



SDEV2150

Lesson 12: Event Handling and State in React

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

In this lesson, students will:

- Implement event handling for user interactions in React components
- Use the `useState` hook to manage and update component state
- Build interactive UI components that respond to user input

Agenda

- Recap: DaisyUI integration from Lesson 11
- React events and the useState mental model
- Demo: Counter component (Hello World of state)
- Controlled inputs and form submission
- Practice: FeedbackForm and peer review
- Exit ticket

CONNECTING

**WE
ARE** ESSENTIAL
TO ALBERTA



What changed since Lesson 11

- We have reusable components
- Styling is consistent with DaisyUI
- Layout is responsive

What we do not have yet is UI that responds to user input.

REACTIVITY

**WE
ARE** **ESSENTIAL
TO ALBERTA**



The idea behind React

State changes.

React re-renders.

UI updates.

Your job is to manage state and let React handle the UI.

QUICK POLL

**WE
ARE** **ESSENTIAL
TO ALBERTA**



What's an example of a user interaction that requires state to update the UI?

- A. Selecting a category
- B. Typing into a search box
- C. Toggling a checkbox
- D. All of the above

REACT EVENTS

**WE
ARE** ESSENTIAL
TO ALBERTA



Events in JSX

```
<button onClick={handleClick}>Click</button>
```

- Declarative, attached in markup
- Bubble like DOM events
- Wrapped by React (synthetic events)

useState

The pattern

```
const [value, setValue] = useState(initialValue);
```

- value is the current state
- setValue schedules an update
- Updating state triggers a re-render

DEMO: COUNTER COMPONENT

Why we start here

- Smallest complete example of state
- Click event updates state
- Render reflects state immediately

Build intuition before applying it to a project.

COUNTER DEMO

**WE
ARE** **ESSENTIAL
TO ALBERTA**




```
const [count, setCount] = useState(0);
```

```
<button onClick={() => setCount(count + 1)}>
```

```
  Incremented {count} times.
```

```
</button>
```

- What causes the re-render?
- What happens if we click fast?

CONTROLLED INPUTS

Controlled means state owns the value

```
<input  
  value={searchTerm}  
  onChange={(e) => setSearchTerm(e.target.value)}  
>
```

- Input reflects state
- Typing triggers events
- Events update state

FORM SUBMISSION

**WE
ARE** **ESSENTIAL
TO ALBERTA**



Override default browser behavior

```
function handleSubmit(e) {  
  e.preventDefault();  
  console.log('Filters submitted');  
}
```

```
<form onSubmit={handleSubmit}>
```

Same concept as before, now attached declaratively.

HANDS-ON PRACTICE

Exercise: Refactor your existing exercise project to make use of the useState hook to add state to the app

Objective: Implement local component state for interactive UIs

- Obtain the exercise starter files
 1. Import the useState hook
 2. Add state to form fields
 3. Ask questions if you are unsure of what to do or how the code works.

GUIDED DISCUSSION

**WE
ARE** **ESSENTIAL
TO ALBERTA**



What happens on state updates?

- State update schedules a render
- React batches updates for efficiency
- Rendering is recalculating UI from state

If something looks wrong, inspect state first.

COMMON MISTAKES

- Forgetting `value` on controlled inputs
- Reading `e.target.value` for checkboxes
- Mutating arrays instead of creating new arrays
- Expecting state to update instantly in the same line
- Trying to update the DOM manually

Lesson Summary

- In this lesson, we covered the following:
 1. Implement event handling for user interactions in React components
 2. Use the useState hook to manage and update component state
 3. Build interactive UI components that respond to user input
- Complete lesson 12 checklist

EXIT TICKET

Submit:

- A short explanation of one component that updates UI using useState
- A small code snippet showing the event handler and state update

Be ready to explain:

- The event
- The state
- The UI change